

4. Hierarchie paměti v počítači (typy a principy pamětí, princip lokality, organizace rychlé vyrovnávací paměti)

4. Hierarchie paměti v počítači

SZZ okruh 4 (FIT BUT). Hierarchie existuje kvůli **poměru cena/výkon**: čím blíže procesoru, tím rychlejší, dražší a menší.

Shrnutí

Dělení a parametry

- Úrovně: **registry** □ **cache (L1/L2/L3)** □ **RAM** □ **SSD/HDD** □ **optická/vnější**.
- Dělení dle: stálosti (**volatilní** × nevolatilní), doby uchování (**statická** SRAM × **dynamická** DRAM s obnovou), přístupu (**RAM** × **SAM**), měnitelnosti (ROM/PROM/EPROM/EEPROM/RWM).
- Parametry: kapacita ($N \times n$ bitů), **přístupová doba**, přenosová rychlost, chybovost, MTBF.

Princip lokality

- **Časová lokalita** — nedávno použitá položka bude pravděpodobně použita znovu.
- **Prostorová lokalita** — sousední položky budou pravděpodobně použity také.
- Lokalita je důvod, proč cache funguje s vysokým **Hit Rate** (95–99 %).

Rychlá vyrovnávací paměť (cache)

- SRAM blízko CPU; přibližné latence: registr 1 cyklus, L1 ~3, L2 ~14, RAM ~240.
- **Hit / Miss, miss penalty** (ztrátová doba). Data se přenášejí po **blocích**.

[[media/szz-04/media/image3.png]] *Přímé mapování cache — maskování horních bitů adresy hlavní paměti.*

[[media/szz-04/media/image1.png]] *Vícecestné (set-asociativní) mapování — více bloků se stejnými spodními bity + příznak.*

- **Mapování**: přímé, **n-cestné**, **plně asociativní** (drahé, pro velké cache nerealizovatelné), skupinově/sektorově asociativní.
- Výběr obětí: LRU, MRU, FIFO, náhodně.

Koherence (konzistence) dat

- **Write-Through** — zápis ihned i do nižší úrovně (jednoduché, pomalejší).
- **Write-Back** — zápis až při odsunu bloku, jen u modifikovaných (**dirty bit**).
- **Write-Buffer** — odsouvaný blok jde do mezipaměti pro zápis, čte se bez čekání.

Souvislosti

Latence cache je klíčová pro [[okruhy/07-princip-cinnosti-pocitace-pipelining|zřetěžené zpracování]] (datové hazardy u LOAD). Volba [[okruhy/05-vestavene-systemy|von Neumann × Harvard]] určuje, zda mají kód a data společný adresový prostor. Buňky SRAM viz [[okruhy/03-sekvencni-logicke-obvody]].

Související syntéza

- [[synthesis/vyhledavaci-b-stromy|Vyhledávací stromy × paměťová hierarchie]] — syntéza

Časté otázky u zkoušky

Na co se u tohoto okruhu typicky ptají. Plné odpovědi níže.

Hierarchie pamětí □ [[#Dělení a parametry]] - *Proč existuje hierarchie?* □ Nelze mít rychlou, velkou a levnou paměť zároveň □ kompromis cena/výkon (čím blíže CPU, tím rychlejší, dražší, menší). - *SRAM vs. DRAM?* □ SRAM = klopné obvody, rychlá, drahá, malá (cache/registry); DRAM = tranzistor + kondenzátor, nutná pravidelná obnova, levnější, větší (RAM). - *Pořadí podle rychlosti?* □ registry □ cache L1/L2/L3 □ RAM □ SSD □ HDD.

Princip lokality □ [[#Princip lokality]] - *Časová vs. prostorová?* □ Časová: nedávno použité se použije znovu; prostorová: použijí se sousední adresy. Díky tomu má cache vysoký hit rate. - *Kdy lokalita nefunguje?* □ při čistě náhodném přístupu do velké paměti.

Cache / RVP □ [[#Rychlá vyrovnávací paměť (cache)]] - *Hit vs. miss, co při miss?* □ Hit = data v cache; miss = nutno načíst blok z nižší úrovně (miss penalty). - *Jak pozná, že data jsou v cache?* □ adresa = tag + index + offset; porovná tag a kontroluje validity bit. - *Druhy mapování?* □ přímé (jedno místo), n-cestné set-asociativní, plně asociativní (drahé); výběr oběti LRU/FIFO/random. - *Write-through vs. write-back?* □ WT zapisuje ihned i do nižší úrovně; WB až při odsunu bloku (dirty bit).

Plné znění (ke studiu)

Paměť – Uchovává data, která je zde možné zapsat a později zase získat (přečíst).

Dělení pamětí

Hierarchie - Hierarchie existuje z důvodu co nejlepšího **poměru cena/výkon**. Stejně tak jsou potřeba jak **volatilní**, tak **nevolatilní** paměti. Obecně platí, že čím **blíže** procesoru paměť je, tím je **rychlejší**, ale také **dražší** (náročnější na výrobu) a má **menší kapacitu**.

- **Vnitřní** uvnitř procesoru: **registry, cache** (RVP, obvykle několik úrovní)
- **Primární**/Hlavní (mimo procesor): **RAM**
- **Sekundární** (uvnitř PC) - **HDD, SSD**.
- **Terciární/vnější** (mimo PC) - **CD, DVD, Flash disk**.

Fyzikální princip - opět výhody a nevýhody cena/volatilita/rychlost:

- polovodičové - bipolární, unipolární, unipolární s floating gate - **SSD**
- magnetické - **diskety, HDD, pásky**,
- optické - DVD, CD, Blue Ray,
- magnetooptické
- molekulární

Stálosti obsahu:

- **Volatilní** - Po odpojení paměťové buňky od el. energie se data ztratí (RAM),
- **Nevolatilní** - Data setrvávají i po odpojení paměti z el. sítě.

Doba uchování informace:

- **Statická** - Drží data libovolně dlouho (dokud jsou připojeny k el. energii). **Velká plocha na čipu** (klopné obvody), **rychlá, drahá, malá kapacita** - používá se pro registry a RVP
- **Dynamická** - Data je třeba po určité době (u DRAM v řádu milisekund) obnovovat, jinak se ztratí. **Pomalejší, levnější** (oproti statické), **menší plocha na čipu** (tvořená tranzistorem a kondenzátorem) - používá se pro DRAM paměti.

Přístup k datům:

- **Libovolný** (RAM - Random Access Memory) - přístupová doba **nezávisí** na umístění paměťové položky.
- **Sériová** (SAM - Serial Access Memory) - přístupová doba **závisí** na umístění v paměti, respektive na aktuální pozici čtecí hlavy (pásky)
- **Smišený** - kombinace výše popsaných přístupů. Např. HDD s více záznamovými povrchy. Výběr povrchu je libovolný, natočení hlavy je sekvenční.

Možnost přístupu/měnitelnost:

- **ROM (Read Only Memory)** - Pouze čtení
- **PROM (Programmable ROM)** - Lze **jednou** zapsat data a poté pouze číst
- **EPROM (Erasable PROM)** - Pro vymazání je potřeba použít externí proces, například pomocí UV záření.
- **EEPROM (Electrically EPROM)** - Lze **vymazat elektronicky**
- **RWM (Read Write Memory)** - Lze **číst i zapsat** do ní (SRAM, DRAM)

Princip lokality

Aplikace pracuje obvykle pouze s **malou částí paměťového prostoru** (data programu nebo samotný kód programu).

- **Časová lokalita** - Pokud procesor používá nějakou položku v paměti, je vysoká pravděpodobnost, že ji bude používat **znovu**. To znamená je vhodné položku uložit, co nejbližší k procesoru.
- **Prostorová lokalita** - Pokud procesor pracuje s nějakou položkou v paměti, potom položky, které jsou umístěny v paměti v **blízkosti této položky**, budou s vysokou pravděpodobností **také použity** (aspoň u dobře napsaného kódu). To znamená, že je vhodné okolní položky uložit co nejbližší procesoru, aby toto očekávané čtení/zápis mohlo proběhnout rychleji.

Parametry paměti

- **Kapacita** - Udává množství dat, která paměť dokáže uložit. Udává se ve tvaru **respektujícím organizaci paměti**, tedy jako součin počtu paměťových míst a délky paměťového místa, tj. **N x n bitů**, např. **16K x 1 bit**
- **Přístupová doba** - doba od **zahájení čtení** (tj. udáním adresy paměťového místa a povelu R) po **získání obsahu** paměťového místa.
- **Přenosová rychlost** - Počet **bitů/bytů**, které paměť přenesla za **jednotku času**, např. 1 Gb/s.
- **Chybovost** - Počet chyb za určitý čas.

- **Poruchovost** - Střední doba mezi poruchami (MTBF - mean time between failure)./

Rychlá vyrovnávací paměť (RVP)

Anglicky **cache** - SRAM blízko procesoru, pomáhá rychlejšímu čtení/zápisu dat, než jaké by bylo použitím pouze operační paměti (RAM - pomalejší DRAM). V procesorech často rozdělena do několika vrstev - L1, L2 pro každé jádro a L3 sdílená mezi jádry. Hierarchické členění pamětí se snaží eliminovat rozdíl mezi rychlostí procesoru a rychlostí operačních pamětí.

Čtení/zápis	Počet cyklů
Registr	1 (i méně při pipelining)
RVP L1	cca 3
RVP L2	cca 14
Hlavní paměť (RAM)	cca 240

RVP je rozdělena do **bloků** o konstantní velikosti, která ideálně odpovídá velikosti bloků v hlavní paměti (RAM), což umožňuje jednodušší přesun dat (po blocích) z RAM do RVP. V RAM je ale paměťových bloků **řádově více** než v RVP, takže musí existovat mechanismy zajišťující, aby v RVP byly **potřebné bloky**. Tyto mechanismy vybírají bloky, které budou do RVP nově přidány a bloky, které v důsledku budou odstraněny. Musí pracovat s velmi **vyšokou pravděpodobností úspěchu**, jinak by paměťová hierarchie zpomalovala přístup k datům.

- **Hit Rate** - udává pravděpodobnost nalezení dat v RVP, v praxi **95-99%**.
- **Miss Rate** (1 - Hit Rate) - pravděpodobnost, že data **nejsou** v cache nalezena. V tomto případě je nutné potřebný blok s daty načíst z hlavní paměti, což obnáší uvolnění místa v RVP, vyhledání bloku v RAM a přenos dat. Tato doba se označuje jako ztrátová doba (**miss penalty**).
- **Přístupová doba** - doba potřebná k nalezení bloku v RVP, blok dat musí být v RVP.
- **Ztrátová doba** - doba, za kterou se musí popřípadě uvolnit místo v cache, nalézt data v hlavní paměti a nahrát je do cache.
- Doba čtení:
 - **Hit** = přístupová doba
 - **Miss** = přístupová doba + ztrátová doba (tato doba je delší, než v případě **nepoužití** RVP, musí k ní docházet minimálně)

Organizace RVP

Je dána zvoleným mapováním. Účel organizace RVP je především co nejvíce snížit **Miss Rate**. Organizace určuje jak jsou bloky z hlavní paměti přiřazovány/mapovány do RVP.

- **Přímé mapování** - Do RVP se mapují bloky na základě adres. Protože adresový prostor RVP je několikanásobně **menší**, než adresový prostor hlavní paměti, **maskují** se u adres hlavní paměti **horní bity**, tak aby délka adresy odpovídala délce adresy v RVP. To znamená, že bloky s adresou se stejnými spodními bity se mapují na jeden blok v RVP a **nemohou** tam tak být umístěny **současně**. Jedná se o jednoduchý koncept, který ale může způsobovat velký počet výpadků (procesor současně pracuje s dvěma bloky, které mají stejnou adresu v RVP).
 - např. pro RVP s 2^{10} bloky a paměti s 2^{30} bloky (protože $2^2 = 4$ B je obsah pole v RVP) bude pravděpodobnost při náhodném výběru bloku $2^{10/2} / 2^{30} = 1/2^{20}$ zaokrouhleně 1/1 000 000, tedy **0.0001%**.

- díky **časové a prostorové lokalitě** není ale přístup do paměti náhodný, proto je v praxi pravděpodobnost **Hit** tohoto řešení výrazně vyšší, okolo **90%**.

[[media/szz-04/media/image3.png]]

- **Vícecestné mapování** - umožňuje do RVP uložit současně více bloků, které mají stejné spodní bity adresy (ukazatel). Adresový prostor RVP je tak rozdělen do více tabulek (2 pro dvoucestné, 4 pro čtyřcestné, ...), u záznamů v těchto tabulkách musí být uložena i zbylá část (maskovaná) adresy - **příznak**, aby se mohl vybrat ten správný záznam pro čtení/zápis. To znamená, že více cestná RVP o stejné **kapacitě dat** jako jednocestná, bude vyžadovat větší celkovou kapacitu a samozřejmě logiku (**komparátor**), která bude provádět **výběr správného záznamu**.
[[media/szz-04/media/image1.png]]
- **Plně asociativní mapování** - K této úrovni lze dospět zvyšováním stupně asociativity, až je tento stupeň roven počtu ukládaných záznamů v RVP (tj. nepoužívá se už adresa bloků RVP). Poté může být libovolný záznam uložen kdekoli v RVP a s ním **celá** jeho adresa (do RAM) jako příznak. Tento způsob uložení ale implikuje to, že musí být zajištěno **souběžné porovnání** adres všech uložených bloků v RVP, aby bylo možné **rychle** najít ten požadovaný. To je ale technologicky náročné (pokud vůbec možné pro velké počty bloků), drahé a v praxi **nepoužitelné**.
[[media/szz-04/media/image2.png]]

Pravděpodobnost výpadku lze samozřejmě nejlépe snížit **zvýšením kapacity** RVP, ale také lze snížit určením vhodné **velikosti bloku**. Větší blok znamená menší objem paměti celkové RVP (nemusí se ukládat tolik adres, lze víc místa použít pro data)

U vícecestných RVP se musí také řešit problém výběru oběti, pokud je RVP pro daný ukazatel (adresu) plná (všechny bloky jsou označeny jako **validní**). Používají se metody jako náhodný výběr, LRU, MRU, FIFO atd. Obecně tyto metody vyžadují další logiku (obvody). [[media/szz-04/media/image4.png]]

- **Skupinově asociativní mapování** - Kompromis mezi přímým a plně asociativním mapováním. Paměť je rozdělena do skupin, kde každá skupina obsahuje stejný počet bloků. Adresa skupiny je vyhledávána přímo, blok v ní pak asociativně.
- **Sektorové mapování** - Hlavní paměť je rozdělena do sektorů, které mají několik bloků. Cache je také rozdělena do sektorů o několika blocích. Sektor hlavní paměti může být uložen do libovolného sektoru v cache, ale bloky musí být v sektoru shodné. Při přenosu bloku do cache jsou tedy staré bloky označeny za neplatné (příznakovým bitem) a je tam blok vložen.

Konzistence dat v RVP

Při zápisu dat do bloku RVP (nejblíže CPU) ztratí bloky nižších RVP a hlavní paměti platnost a **nesmí** se již použít, musí se nějak označit. Problém je zejména zjevný u více jádrových procesorů (např. každé jádro má vlastní L1 a L2 cache, sdílí L3). Zápis do L1 jednoho jádra musí případně nevalidovat blok v L1 druhého jádra, které si jej musí znovu načíst (problém souběžného přístupu řeší programátor).

Metody zaručení konzistence:

- **Write-Through** (přímý zápis) - Při přímém zápisu do RVP se zapisuje okamžitě i do bloku v hlavní paměti. Tento princip je jednoduchý na realizaci, ale při častém zápisu je pomalý (musí se často čekat na pomalou RAM). Lze poté rozdělit:
 - **Write-Through with Write Allocate** - Write-through, pokud ale nejsou zapisovaná data v cache, dojde k jejich načtení.
 - **Write-Through with no Write Allocate** - Nedojde k načtení zapisovaných dat do cache.
- **Write-Back** (zpětný zápis) - K úpravě (zapsání) bloku do nižší úrovně paměti (hlavní paměti) dochází, až když je blok z RVP **odstraněn** (odsunut). Neefektivní, protože k zápisu dochází **vždy**,

i když **nedošlo** ke změně (musí se čekat). Proto se bloky označují příznakem změny (**dirty bit**) a zápis pak probíhá následovně:

- **Flagged Write Back** - Zápis do paměti se uskuteční až při uvolnění bloku z cache, ale **pouze** u těch bloků, které byly modifikovány (mají **nastavený dirty bit**).
- **Flagged Register Write Back** - Opět jsou ukládány pouze modifikované bloky, ale ne přímo, jsou prvně zapsány do pomocného rychlého bloku - zápis je odložen a je možné tedy číst bez čekání. **Nejrychlejší**, ale **nejnáročnější** (nejdražší) způsob.
- **Write-Buffer** (zápis s mezipamětí) - rozšiřuje metodu **Write-Back** o to, že při vybrání oběti (bloku, který bude odsunut z RVP s nastaveným dirty bitem), se data bloku přesunou do mezipaměti pro zápis, ze které budou zapsány, až nebudou požadavky na čtení z hlavní paměti. Nemusí se tak čekat na zápis dat, která jsou odsouvána z RVP předtím, než budou nahrazena novým blokem.

Zdroje

- SZZ okruh 4 — studijní materiály FIT BUT (szz-04.docx). Další obrázky: media/szz-04/.

Navigace: [[index| • Přehled okruhů]] · □ [[okruhy/03-sekvenční-logické-obvody|3. Sekvenční logické obvody]] · další: [[okruhy/05-vestavěné-systémy|5. Vestavěné systémy]] □